

C: K5120、GateUp/SwiGLU 实现和构建

本文由总览原第 6、7.1、7.4 节拆出，作为 C 的独立实现、构建和专项验收手册。全局基线、Qwen layer 命中、三人调用关系、服务冒烟和合并标准见 [Qwen3.5 优化分工总览](#)。

1. 实现

1.1 实现范围

C 是 `gfx936.py` 的文件 owner，先提供缓存的公共设备判断

`torch.cuda.get_device_properties(...).gcnArchName.startswith("gfx936:")`，供A的attention 公共gate（间接保护B的page784）、A的GDN和C的K5120窄gate共用。B不直接导入该helper。该helper只识别设备，不混入模型shape或功能条件。

C 复用官方 `_rocm_C.LLMM1` ABI，不新增 extension 或 binding。Python helper 只在以下条件接管：输入 K=5120、总共一个 5120 维向量、weight/input BF16、weight contiguous、input 最后一维 stride 为 1、gfx936，且输出 M 属于 `{96,14336,16384,34816,248320}`。通用线性入口还要求 `bias is None`。

native kernel 使用 640 threads: 5120/8=640 个 `float4` chunk。后 320 lanes 写 LDS，前 320 lanes 合并后由 5 个 wave leader 写第二级规约，最终得到 FP32 sum 并写 BF16。M=96 每 CTA 处理 4 行，其余普通 shape 每 CTA 处理 2 行。

GateUp/SwiGLU 只接管 `(34816,5120)` 且 `expert_gate is None`：

1. 第一遍计算 17408 行 gate，暂存在输出 buffer。
2. 第二遍计算 17408 行 up。
3. 第二遍写回前执行 BF16-staged `SiLU(gate) * up`，输出 shape 为 17408。
4. 随后调用官方 `down_proj`；本分支不优化 `(5120,17408)` 的 K17408 down projection。

任何 Python gate 不满足时 helper 返回 `None`。MLP 回到 `Qwen2MoeMLP.forward`；通用 linear 回到官方 ROCm GEMM 分派；`LLMM1` 的其他 shape 继续走官方 `LLGemm1_kernel`。

C 与 A/B 的协作和解耦

- **C和A的共享文件边界**：C是 `gfx936.py` 文件owner，负责import、公共 `is_gfx936()` 和K5120 helper；A是GDN Norm+SiLU函数段的语义owner。两人避免同时改文件 骨架，合并时由C处理文本冲突、A确认GDN数学和 fallback没有变化。
- **公共设备门的影响面**：A的attention公共gate直接调用 `is_gfx936()`，从而间接决定B的 page784能否进入；A的GDN和C的K5120也直接调用它。C修改缓存或设备识别后，必须通知A 并重跑A的GQA6/GDN以及B的page784测试，不能只测GEMV。
- **C和B的边界**：没有直接代码或ABI依赖。B使用预装FA/Triton JIT，C使用本仓库 `_rocm_C` HIP extension；任何一方的kernel内部调整都不要求另一方改实现。
- **构建协作**：C的native修改决定最终 `.so`，因此C的提交应在最终wheel构建前合入；A/B可先做源码树内Triton专项测试，但最终三人都必须从源码树外验证同一个新wheel，避免 新Python配旧 `.so`。C不能在构建过程中替换预装FA、Triton或PyTorch。
- **依赖边界**：C只复用OpenDAS仓库已有 `LLMM1` binding、HIP源码和比赛容器工具链，不增加新extension、第三方kernel仓库或pip包。

1.2 以官方源码为模板的改法

C 应围绕官方已有 ABI、GEMM 分派和 MLP 类做小改动，不能另建一套 extension：

- 官方 ROCm `skinny_gemms.cu`
- 官方 ROCm unquantized GEMM 分派
- 官方 ROCm platform/device capability
- 官方 Qwen2MoeMLP
- 官方 Qwen3.5/Qwen3Next model
- 官方 `setup.py`

Native：在官方 LLMM1 内插 shape-specialized launch

1. 从官方 `LLGemm1_kernel` 复用 BF16/FP16 类型转换、weight/input 读取、wave shuffle、stream 和输出分配方法。K5120 kernel 只替换固定 K 下的线程/规约组织。
2. 保留官方 `LLMM1` 函数签名和 pybind ABI。shape 判断放在官方 `TORCH_CHECK` 之后，不新增 `_rocm_C2`、新 binding 或新的 Python op 名称。
3. 只为 fused SwiGLU 把 `out_c` 最后一维改成 `M/2`；普通 shape 仍使用官方 `M`。
4. 在官方 `AT_DISPATCH_REDUCED_FLOATING_TYPES` 前 early return 专用 kernel；整个官方 dispatch block 原样留在函数尾部，承担 FP16、其他 K/M 和 rows-per-block fallback。
5. fused GateUp 的数学顺序以官方 `Qwen2MoeMLP.forward` 为规范：`gate_up_proj -> SiluAndMul -> down_proj`。native 只把前两步融合，`down_proj` 仍调用 官方线性层。

Python：在官方入口前尝试，失败继续原函数

1. 先按官方 `RocmPlatform` 读取的 device properties 建一个缓存的 `is_gfx936()`；A 的 GDN 和 attention 公共 gate 只导入这个公共门，B 的 page784 由该 gate 间接保护，三处 都不复制设备识别或全局状态。
2. 从官方 `rocm_unquantized_gemm_impl` 开始，在读取 `n/m/k` 后加入 `bias is None` 的 helper 调用；helper 返回 `None` 后，后续 CU 计算、AITER、skinny GEMM 和 `F.linear` 不改。
3. 不复制 `Qwen2MoeMLP.__init__`。继承官方类，只覆盖 `forward`：先尝试 fused GateUp；成功后调用已有 `self.down_proj`，失败执行 `super().forward(x)`。这样 weight loader、TP sharding、activation 和 expert gate 语义都来自官方类。
4. K5120 的五个 M 值从官方 Qwen 构造函数推导：`Qwen3NextAttention.qkv_proj`、`Qwen3_5GatedDeltaNet.create_qkvz_proj/create_ba_proj`、`Qwen2MoeMLP.gate_up_proj` 和 `ParallelLMHead`。不要在 kernel 中支持没有 Qwen layer 调用证据的额外 M。
5. 构建只恢复官方 `setup.py` 已预留但注释的 `vllm._rocm_C` CMakeExtension；CMake source/binding 继续使用 仓库原文件。

最小 Python 改造结构是：

```
def official_rocm_gemm(...):
    if bias is None:
        result = qwen35_k5120_gemv(weight, x)
        if result is not None:
            return result
    # 官方 ROCm GEMM 分派保持不变。

class Qwen3NextMLP(Qwen2MoeMLP):
    def forward(self, x):
        result = try_fused_gate_up(x)
        return self.down_proj(result)[0] if result is not None else super().forward(x)
```

1.3 C 的代码索引

Path	当前行号	内容介绍	可参考的官方代码（fa718036）
<code>csrc/rocm/skinny_gemms.cu</code>	234-342	文件调用链、BF16x8 dot、640-thread 两级规约、可选 fused SiLU*up	同文件 <code>LLGemm1_kernel</code> 的加载、规约和 BF16 输出方式
同上	344-403	在官方 <code>LLMM1</code> 检查后识别 K5120 shape，选择 2/4 rows 或 fused 两次 launch；未命中继续官方 dispatch	同文件官方 <code>LLMM1</code> 与 <code>AT_DISPATCH_REDUCED_FLOATING_TYPES</code> fallback
<code>vllm/model_executor/layers/fla/ops/gfx936.py</code>	1-81	两条调用链、共享 gfx936 设备门、GDN 融合、K5120 shape/dtype gate、 <code>LLMM1</code> 调用和 fallback	<code>vllm/platforms/rocm.py</code> 的设备识别； <code>vllm/model_executor/layers/</code> <code>utils.py::rocm_unquantized_gemm_impl</code>
<code>vllm/model_executor/layers/</code> <code>utils.py</code>	122-140	在官方 ROCm unquantized GEMM 开头尝试无 bias K5120 fast path	同文件官方 <code>rocm_unquantized_gemm_impl</code>
<code>vllm/model_executor/models/qwen3_next.py</code>	37-41、79、112-126	导入 gfx936 helper；用可 fallback 的子类覆盖 dense <code>Qwen3NextMLP.forward</code>	<code>vllm/model_executor/models/qwen2_moe.py::Qwen2MoeMLP.__init__</code> /forward
<code>setup.py</code>	997-1001	构建调用链注释；HIP 构建时启用官方 <code>vllm._rocm_C</code> extension	官方 <code>setup.py</code> 的 <code>CMakeExtension/ext_modules</code> 组织方式

1.4 C 的验收

- 普通 GEMV 分别测试 `M=96/14336/16384/34816/248320`，与 `F.linear` 比较 finite、max/mean error 和 allclose。
- fused (`34816,5120`) 必须与官方 `SiluAndMul(F.linear(x, gate_up_weight))` 比较，并验证输出 shape 为 17408。
- 测试非 gfx936、FP16、非连续 weight、`x.numel()!=5120`、unsupported M、bias 和 `expert_gate is not None` 的 fallback。
- 单独验证公共设备门在 gfx936 返回 true，在 CPU/非 gfx936 返回 false；A/B 不得再有 第二份设备识别实现。
- 必须重新构建 `_rocm_C.abi3.so` 后再测；仅修改 Python 或复用旧 `.so` 不算验收。

2. 构建与专项测试

2.1 完整 wheel 构建与隔离安装

C 使用下面这组已经在当前比赛容器执行成功的命令。构建放在临时 worktree，是因为 OpenDAS `setup.py` 会重写源码树中的 `vllm/version.py`；`--target` 安装不会覆盖容器依赖。

```
cd /public/home/tangyu408/Qwen_DCU_Worker_0/pr2026-bh408-gqa-page784-k5120
git worktree add --detach /tmp/qwen35-build-source HEAD
cd /tmp/qwen35-build-source
mkdir -p /tmp/qwen35-build/{dist,bdist,cache}

VLLM_TARGET_DEVICE=rocm MAX_JOBS=16 python3 setup.py \
  build --build-base /tmp/qwen35-build/build \
  bdist_wheel --bdist-dir /tmp/qwen35-build/bdist \
  --dist-dir /tmp/qwen35-build/dist
```

```
python3 -m pip install --no-deps --target /tmp/qwen35-build/site \
/tmp/qwen35-build/dist/vllm-*.whl
```

后续命令统一使用：

```
REPO_ROOT=/public/home/tangyu408/Qwen_DCU_Worker_0/pra2026-bh408-gqa-page784-k5120
ARTIFACT_ROOT=/tmp/qwen35-build
SITE_DIR=/tmp/qwen35-build/site
CACHE_ROOT=/tmp/qwen35-build/cache
cd "$REPO_ROOT"
```

2.2 K5120 与 GateUp/SwiGLU 专项检查

C 修改 `csrc/rocm/skinny_gemms.cu` 后必须执行本文件 2.1 的完整 wheel 构建；只设置 `PYTHONPATH=.` 会得到新 Python 配旧 `.so`，不能作为测试。最短检查与安装后测试：

```
OFFICIAL=fa718036bdb9dfd80a872b86c8ac16c9d02bfd31
git clang-format --diff "$OFFICIAL" -- csrc/rocm/skinny_gemms.cu | \
  grep -Fx 'clang-format did not modify any files'
ruff check vllm/model_executor/layers/fla/ops/gfx936.py \
  vllm/model_executor/layers/utils.py vllm/model_executor/models/qwen3_next.py
CHECK_SCRIPT="$PWD/docs/cscv/verify_qwen35_optimizations.py"
RUN_DIR="$(mktemp -d /tmp/qwen35-k5120-check.XXXXXX)"
(cd "$RUN_DIR" && HIP_VISIBLE_DEVICES=0 PYTHONPATH="$SITE_DIR" \
  python3 "$CHECK_SCRIPT" k5120 swiglu)
```

非代码步骤：确认测试日志打印的新 `_rocm_C.abi3.so` 路径；修改 native kernel 后必须重建 wheel，且 `M=96/14336/16384/34816/248320` 要逐项验收。GateUp还要单测 `fuse_silu=True` 的17408维输出，不能只验证普通 GEMV。

3. 联合验证入口

完成本文件的专项检查后，按 [总览第 7.5 节](#) 继续执行同一新 wheel 的 DP1/DP2 小数据服务冒烟；实测版本、结果、合并顺序和故障备注 统一保留在总览第 7.6--7.8 节。